

Technical Project Documentation

The Autonomous Compliance Sentinel

Project	The Autonomous Compliance Sentinel
Modul	Responsible AI and Data Ethics
Authors	Group 2: Singh, Vikrant and Müller, Kay
Data source	proposals_1000_EN.csv
Subject	Goal 1 to 4
Version	1.0

Table of Contents

<i>Technical Project Documentation</i>	1
<i>The Autonomous Compliance Sentinel</i>	1
1. Assignment	3
1.1 Task description	3
1.2 Planned architecture	3
1.3 Final architecture in detail	3
System context (the pipeline Goal 1 to Goal 4).....	3
Runtime, the LangGraph state machine	4
Inside the assess node	5
A proposal as a sequence (Signal 1 → Signal 2 → human)	6
Component and package map.....	6
2. Week 1 Goal 1	7
2.1 Sources	7
2.2 The policy catalogue RAI-01 to RAI-09.....	7
Sentences	7
2.3 Generator	8
2.4 Characteristics of the dataset	8
2.5 Exploratory data analysis	9
2.6 Project plan	10
2.7 Regulatory analysis	10
2.8 Conclusion Week 1 Goal 1	10
3. Week 2 Goal 2: Baseline model, risk and fairness	11
3.1 Setup.....	11

3.2 Results	11
Threshold set at 0.40.....	11
3.3 Recall per policy.....	11
3.4 Fairness analysis.....	12
Equalized-Odds- Distances on the primary model	12
3.5 Risk analysis	12
3.6 Alternative models explored (XGBoost and embeddings)	13
4. Week 3 Goal 3: Explainability, weakness tests and coverage.....	14
4.1 Explainability findings (XAI)	14
4.2 Automated weakness tests	14
Evasion (masking or obscuring trigger terms)	14
Negation gap (negation_gap / negation_blind_count)	14
4.3 Test coverage (>= 80 %).....	14
4.4 One source of truth: notebook imports the tested package	15
5. Model improvement experiments and methods at the data level	16
Experiment 1: different vectorisation and tokenisation	16
Result:	16
Experiment 2: semantic embeddings instead of TF-IDF	17
Result:	17
Five effective methods at data level (Goal 1)	17
Method 1: Incorporate non-conformities into the requirements text rather than listing them separately	17
Method 2: Paraphrased variance per policy (data augmentation)	18
Method 3: No tell-tale prefix in 'grounded' cases	18
Method 4: Allow implicit and subtle breaches	18
Method 5: A separate test set comprising real, marker-free suggestions	18
5. Implementation Version 2.....	18
Results on the optimised (marker-free) dataset.....	18
6. Week 4 Goal 4: The Autonomous Compliance Sentinel Agent	20
Architecture of the agent, the LangGraph state machine	20
Step by step, a proposal runs through the agent.....	20
Features implemented	22
Offline and fallback behaviour	22
List of tables	24

1. Assignment

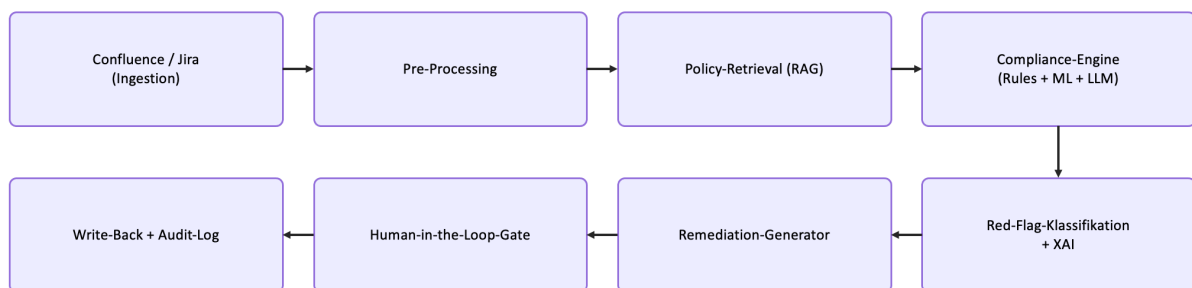
1.1 Task description

Agent that autonomously monitors Confluence spaces to ensure no project documentation violates new transparency or safety requirements:

- Goal: Analyse project proposals vs. internal policies
- Download the data from:
 - <https://www.kaggle.com/datasets/cesaranasco/jira-dataset>
 - And generate 1000 "Internal Project Proposals"—1/3 of those should contain "Ethical Red Flags"
 - Create some internal policies, f.e. from the area of responsible AI
 - See if your Agent catches those "Red Flags"
 - Make the agent suggest fixes for the project and let it write back into confluence/Jira
- Build a responsible AI-Agent system, analyze your data etc.
- Show your results in a documented jupyter-notebook as a presentation

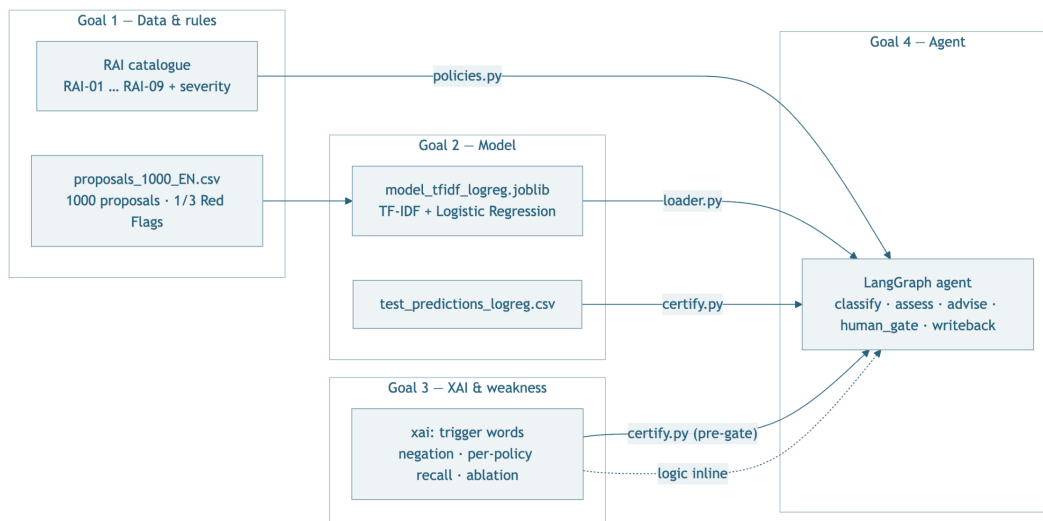
1.2 Planned architecture

Goal 1 has already planned and documented the agent's complete architecture:



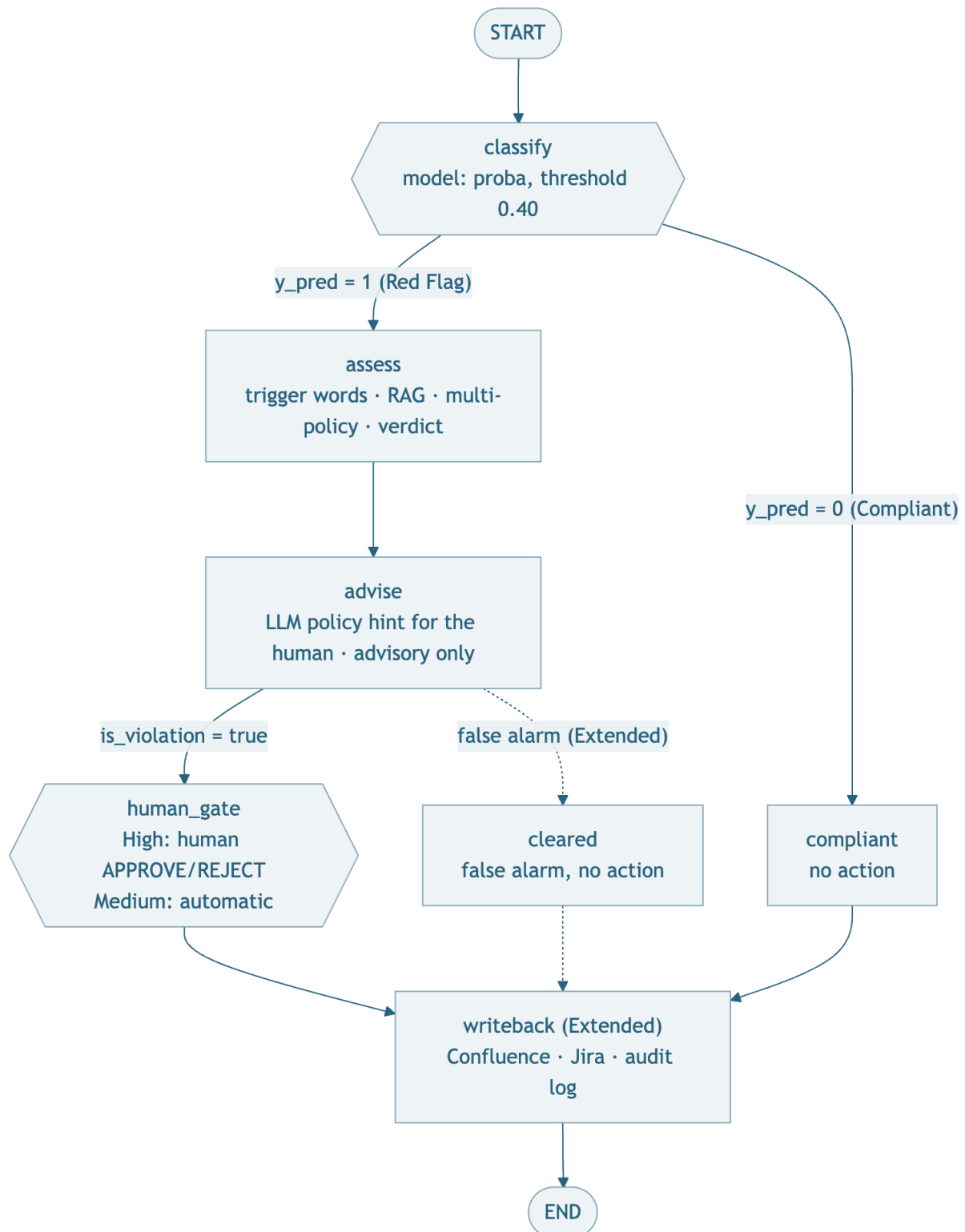
1.3 Final architecture in detail

System context (the pipeline Goal 1 to Goal 4)

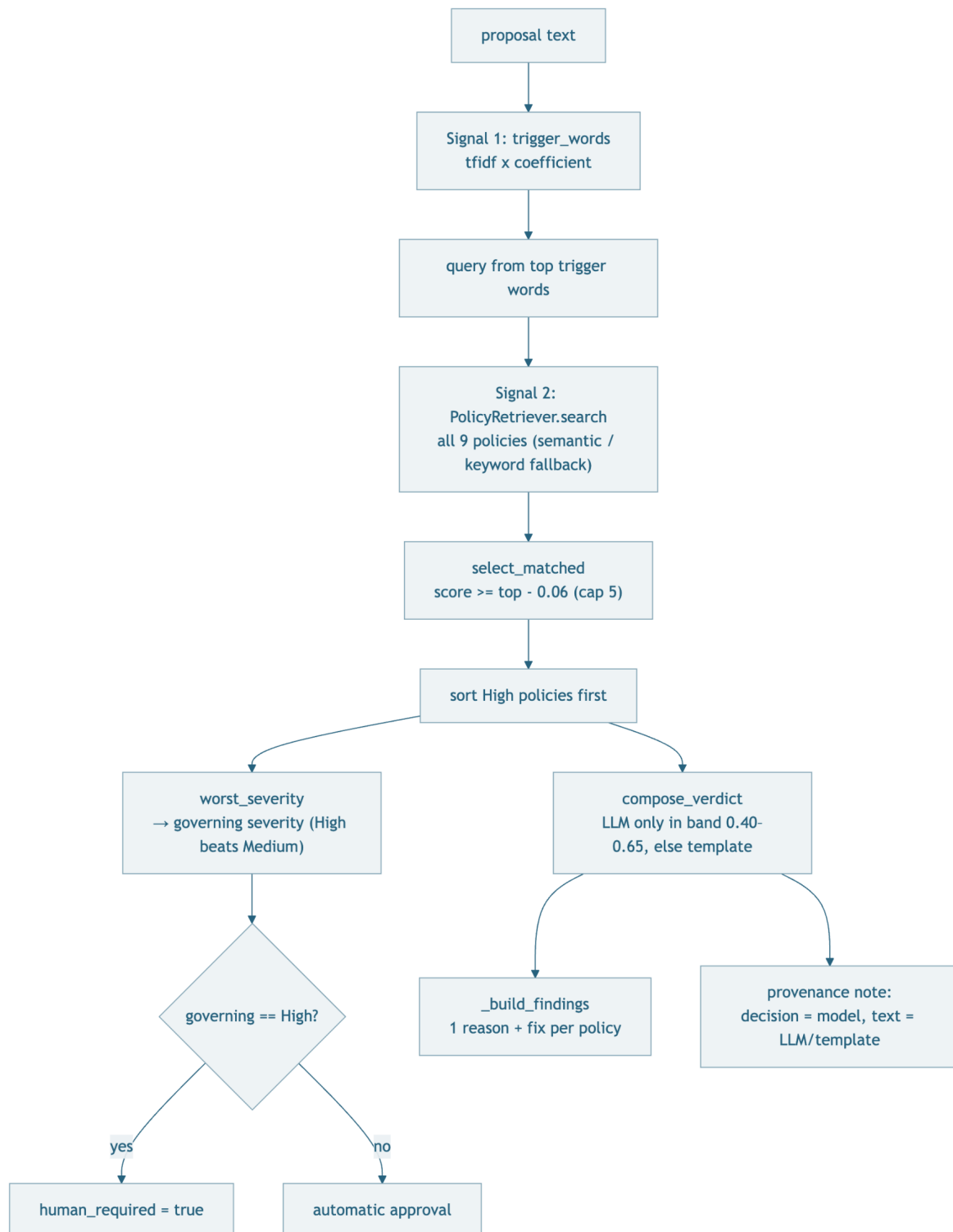


Runtime, the LangGraph state machine

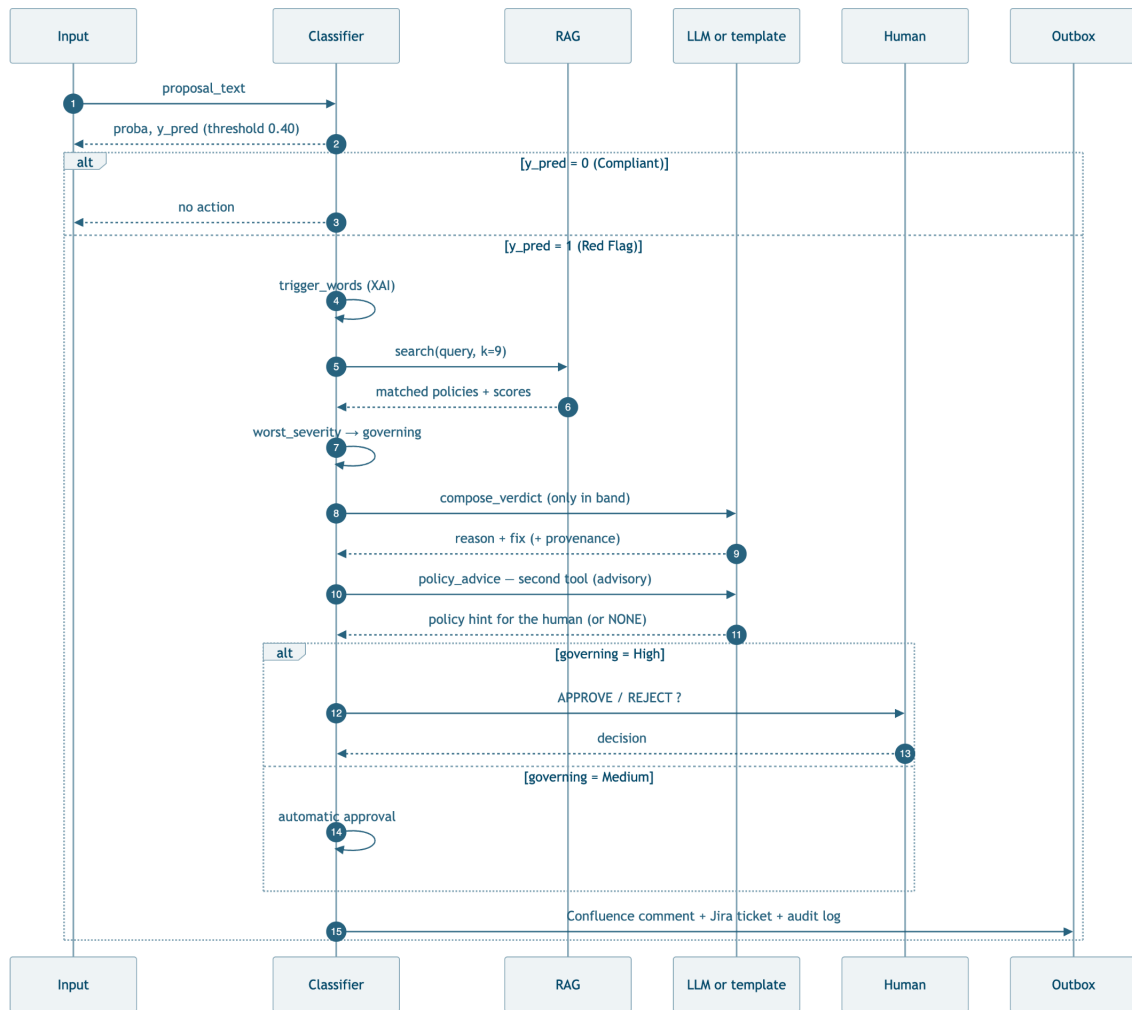
A single proposal runs through fixed, deterministic nodes.



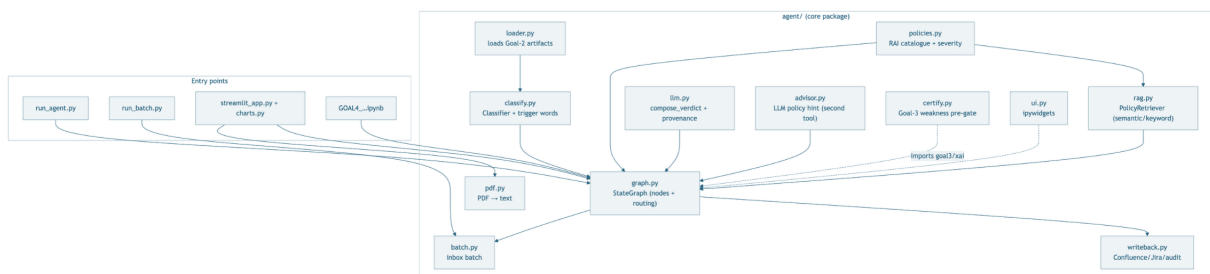
Inside the assess node



A proposal as a sequence (Signal 1 → Signal 2 → human)



Component and package map



2. Week 1 | Goal 1

Requirement for goal 1 is to generate 1000 internal project proposals, one third of them with ethics violations, plus a self-made policy catalogue.

2.1 Sources

File	Origin	Size	Use
GFG_FINAL.csv	Kaggle Jira dataset	114 MB	Provides the structure and vocabulary of real tickets
PROMISE_exp.csv	PROMISE requirements	112 KB	Provides real requirement sentences

Table 1: Datasets

2.2 The policy catalogue RAI-01 to RAI-09

The catalogue is manually made and oriented on the topic areas of the EU AI Act. Each policy consists of two or three fixed violation sentences (19 in total).

RAI-ID	Name	Severity	Sentences	Lecture Notes and EU AI Act ¹
RAI-01	Data Protection	High	2	Pillar Privacy (Week 1) + Art. 10, 13
RAI-02	Transparency	High	2	Pillar Transparency (Week 1) + Art. 10, 13
RAI-03	Fairness	High	2	Pillar Fairness + Art. 10, 13
RAI-04	Human Dignity	High	2	Art. 5 (b)
RAI-05	Prohibited Purpose	High	3	Art. 5 (a), (c) and (f)
RAI-06	Security	Medium	2	Pillar Robustness (Week 1) + Art. 15
RAI-07	Human Oversight	Medium	2	Art. 14
RAI-08	Data Minimization	Medium	2	
RAI-09	Explainability	Medium	2	Pillar Explainability (Week 1)

Table 2: The policy catalogue.

Sentences

RAI-01 Data Protection [High] (real (NFR negated))

“No legal basis for the data processing is documented.”

“Personal data is used without consent.”

RAI-02 Transparency [High] (synthetic)

“The automated decision is not disclosed to data subjects.”

“Users are not informed about the use of AI.”

RAI-03 Fairness [High] (synthetic)

“The target group is selected by gender.”

“Applicants are pre-filtered by origin and age.”

RAI-04 Human Dignity [High] (synthetic)

¹ supplemented in some cases upon receipt of the sets of slides

“The system deliberately exploits vulnerabilities of protected groups (e.g. children, patients).”
“Vulnerable groups are processed without special safeguards.”

RAI-05 Prohibited Purpose [High] (synthetic)

“The system rates people based on their social behaviour (social scoring).”
“Subliminal techniques are used to influence behaviour.”
“Employees' emotions are automatically recognised in the workplace.”

RAI-06 Security [Medium] (real (NFR negated))

“No security or access control is foreseen.”
“The system goes to production without a security assessment.”

RAI-07 Human Oversight [Medium] (synthetic)

“The decision is made fully automatically without human intervention.”
“No human-in-the-loop is foreseen.”

RAI-08 Data Minimization [Medium] (synthetic)

“More data fields are collected than needed for the purpose.”
“Data is stored on stock for unknown purposes.”

RAI-09 Explainability [Medium] (synthetic)

“The model is a black box without any means of justification.”
“Decisions cannot be traced or explained.”

In addition, implementation variant B (marker for a genuine PROMISE requirement):

This requirement is NOT met: "<genuine requirement>"

2.3 Generator

The Generator is a function in the Goal 1 Notebook.

Process for each request:

1. Copy the structure and vocabulary from a real Jira ticket.
2. Incorporate a violation with a probability of 1/3.
3. Variation A: append one or more of the 19 fixed sentences.
4. Variation B: wrap a real requirement from PROMISE with the marker “This requirement is NOT met:”.
5. Log the violated guidelines in the red_flags column.

Note: The generator is deterministic (seed=42, np.random.default_rng) and is therefore reproducible.

2.4 Characteristics of the dataset

Property	Value
Rows	1000
Columns	11 (the column y is added only on loading)

Compliant	667
Red Flag	333 (matches the required quota of 1/3)
grounded_flag = 0	884
grounded_flag = 1	116
Marker rows	116 (in Red-Flag rows)

Table 3: Characteristics of proposals_1000_EN.csv

2.5 Exploratory data analysis

The basis for this is the 1,000 applications submitted.

1. Class balance

667 compliant applications versus 333 red-flag applications, a ratio of 2 to 1. This slight imbalance suggests that it is better to consider precision, recall and F1 rather than simply accuracy.

2. Breakdown of types of violations

The nine categories, RAI-01 to RAI-09, occur with roughly the same frequency, ranging from 66 to 79 instances (evenly distributed). Two of these, RAI-01 (Data Protection) and RAI-06 (Security), are based on real, negated NFRs. The remaining seven are synthetic.

3. Grounding real versus synthetic

Of the 333 red-flag requests, 116 (around a third) contain at least one genuine, negated NFR requirement from the ‘Security’ or ‘Legal’ categories. The remaining 217 are synthetically generated. The data is therefore partly real and not purely synthetic.

4. Text length

Red-flag applications are slightly longer in the middle range, but the ranges overlap considerably (compliant: 33 to 166; red flag: 51 to 193). Text length is therefore not a reliable predictive variable, which argues in favour of a semantic approach.

5. Bias check (using a fictitious group “AI method”)

The red-flag rate varies across the seven AI methods between 0.293 (LLM chatbot) and 0.362 (clustering). There is no systematic bias by method.

6. Data quality

There are no duplicate entries – all proposal_ids are unique. The 667 missing values are found exclusively in the red_flags column and relate precisely to those compliant applications where, as expected, this column is empty.

2.6 Project plan

Week	Work package	Milestone
1	Data (Jira + NFR) & EDA	Data set + analysis
2	Baseline + Risk + Fairness	Benchmark metrics
3	XAI + Tests ($\geq 80\%$)	Evaluation report
4	LLM-Agent + RAG + Write-Back	End-to-end demo
5	Documentation + Pitch + Rehearsal	Final delivery

Table 4: Project plan

2.7 Regulatory analysis

Which frameworks govern a system like this:

Framework	Status	Relevance to the Sentinel
EU AI Act	Binding law	Supporting role, not high-risk as designed; the duties that apply are transparency and human oversight (the Human-in-the-Loop-Gate).
GDPR / DSGVO	Binding law	Legal basis for processing, data minimisation, data subjects' rights - the very themes several RAI policies encode (RAI-01, RAI-08).
ISO/IEC 42001²	Voluntary standard	AI management system: governance and risk processes around the model.
ISO/IEC 23894	Voluntary framework	AI-specific risk management, feeding the Week-2 risk analysis.
NIST AI RMF	Voluntary standard	The Govern-Map-Measure-Manage loop as an operating model for the whole lifecycle.

Table 5: Regulatory analysis

The “no high risk” classification applies to the planned system, i.e. where every change is approved by one person. It is based on the ‘human-in-the-loop’ principle (without this oversight, the risk profile would be higher).

2.8 Conclusion Week 1 | Goal 1

With the completion of Goal 1, the data foundation is now in place:

- 1,000 requests based on a genuine Jira structure and genuine PROMISE/NFR text.
- Some red flags are based on real factors (security, legal), whilst the rest are synthetic.
- Slight class imbalance (Recall: a missed violation is more costly than a false alarm)
- Text length is only a weak indicator, hence a semantic model.
- No structural bias according to the AI method.
- The labelled dataset is saved as proposals_1000_EN.csv and forms the handover to Week 2.

See also: **GOAL1_Autonomous_Compliance_Sentinel.ipynb** | **Goal1_FIN.pptx**

² ISO/IEC 42001:2023 (not in the lecture)

Source: https://blog.codacy.com/iso-42001-what-engineering-leaders-need-to-know-about-the-ai-management-system-standard?utm_term=&utm_campaign=Search+-+DSA&utm_source=adwords&utm_medium=ppc&hsa_acc=9882323101&hsa_cam=23945499275&hsa_grp=194494740861&hsa_ad=813114651986&hsa_src=g&hsa_tgt=dsa-2492021383621&hsa_kw=&hsa_mt=&hsa_net=adwords&hsa_ver=3&gad_source=1&gad_campaignid=23945499275&gbraid=0AAAAADGVzoEt0NOKGIOW3j32GF4Vao-Rp&gclid=Cj0KCQjwgULSBhDLARiAH-yPrHL4mlevPWZbV2fXCr0FAARqfDbY_Pkjk6ELr8uHvC8D0rAndVj7kaAplMEALw_wcB

3. Week 2 | Goal 2: Baseline model, risk and fairness

3.1 Setup

The model is implemented as a pipeline in the sentinel package.

Component	Setting
Vectorisation	TF-IDF, ngram_range = (1, 2), stop_words = "english"
Features	8083
Primary model	Logistic Regression, class_weight = "balanced"
Intercept	-0.664 (Bias term in logistic regression)
Threshold	0.45 (lowered in favour of recall)
Comparison model	Multinomial Naive Bayes
Optional	XGBoost (not in the default factory, no artifact shipped)

Table 6: Setup

A missed violation is more costly than a false alarm, which is why recall is of significant importance for this project. A false alarm is noticeable during manual inspection, whereas a missed violation is not.

3.2 Results

Metric	Value
Test set size	200 rows
of which Red Flag	67
Recall	0.925
Precision	0.939
F1	0.932
Decision threshold	0.45
Missed violations (FN)	5
Over-flagged (FP)	4

Table 7: Results on the test set

The threshold was lowered to catch more violations, in line with the recall focus.

Threshold set at 0.40

Lowering the threshold to 0.40 raises the recall to 0.970 (only 2 missed violations), but reduces the precision to 0.855. However, this results in 11 false alarms. The value of 0.40 is kept in reserve, in favour of recall.

3.3 Recall per policy

RAI-ID	Name	n	Recal
RAI-01	Data Protection	18	0.889
RAI-02	Transparency	16	0.938

RAI-03	Fairness	17	0.882
RAI-04	Human Dignity	18	1.000
RAI-05	Prohibited Purpose	11	1.000
RAI-06	Security	15	0.933
RAI-07	Human Oversight	11	1.000
RAI-08	Data Minimization	14	1.000
RAI-09	Explainability	15	1.000

Table 8: Recall per policy

3.4 Fairness analysis

The dataset contains no demographic features and there are no columns for gender, origin or age, because the proposals are project documents (not describe persons). The three available grouping features were therefore examined.

Group	n	Recall
grounded_flag = 0	42	0.929
grounded_flag = 1	25	0.920

Table 9: Group – GAP 0.009

issue_type	n	Recall
Bug	59	0.915
Suggestion	8	1.000

Table 10: issue type – GAP 0.085

ai_method	n	Recall
Recommender	13	0.846
Scoring	12	0.917
Computer Vision	11	1.000
LLM Chatbot	10	0.800
Clustering	9	1.000
Forecasting	8	1.000
Classification	4	1.000

Table 11: 7 Groups

Equalized-Odds- Distances on the primary model

The notebook shows TPR distances of 0.009 (grounded_flag), 0.085 (issue_type) and 0.200 (ai_method). The AI method distance of 0.200 is the largest, but relates to very small subgroups (including, amongst others, only four positive cases per method).

3.5 Risk analysis

A risk register was drawn up and assessed in terms of probability of occurrence and extent of damage.

R-ID	Risk	Likelihood	Impact
R1	Missed violation (false negative)	High	High

R2	Over-flagging (false positive)	Med	Med
R4	Synthetic-data leakage / shortcut learning	High	High
R5	Data / concept drift	Med	High
R6a	Model evasion (classifier)	Med	High
R6b	Prompt injection (LLM agent)	Med	High
R7	Automation bias / over-reliance	Med	High
R8	Group performance gap (fairness)	Med	High

Table 12: 7 The risk list as defined in Week 2

3.6 Alternative models explored (XGBoost and embeddings)

Model	Recall	Precision	F1	ROC-AUC	Assessment
TF-IDF + Logistic Regression (primary)	0.925	0.939	0.932	0.991	The chosen model; transparent (0.964 recall in 5-fold cross-validation).
TF-IDF + XGBoost	0.970	0.956	0.963	0.997	Higher recall (0.985 in cross-validation, wins CV), but this reflects not better understanding.
XGBoost + BGE-small embeddings	0.657	0.710	0.682	0.868	Semantic features instead of literal tokens; recall collapses.

Table 13: Alternative models tested in Week 2

LogReg remains the primary model. It directly returns the trigger words for each decision via linear weights (top_terms) and the subsequent objectives require explainability (XAI, justifying agent). Linear weights are also reproducible and human-readable (EU AI Act, Article 12 on record-keeping and Article 14 on oversight).

See also: **GOAL2_Autonomous_Compliance_Sentinel.ipynb** | **Goal2_FIN.pptx**

4. Week 3 | Goal 3: Explainability, weakness tests and coverage

In Goal 3, we analyse the baseline (TF-IDF + logistic regression) from Goal 2 with a view to explainable AI and automated vulnerability testing. The reusable logic is contained in the importable Python package (xai/). This enables the use of a test suite that includes measurable code coverage.

4.1 Explainability findings (XAI)

The linear weights are dominated by a single marker (template), which is equivalent to a label leak. The token “met” / “requirement met” (the fixed prefix “This requirement is NOT met:”) has a coefficient of 2.577. An ablation test confirms that removing a single marker has virtually no effect on recall. However, removing the entire marker family reduces it from 0.991 to 0.586.

Key finding: The model recognises generator templates, but not ethical content.

With `stop_words="english"` sklearn removes "not / no / without / cannot" before the model sees them, so that "with consent" and "without consent" become the same set of tokens. On five minimal pairs, the baseline is completely blind (5/5, gap 0.000); the POS tokeniser is only partially helpful (mean gap 0.079).

4.2 Automated weakness tests

The Goal-2 risks are converted into repeatable pass/fail tests in `xai/weakness.py`.

Evasion (masking or obscuring trigger terms)

Recall drop of just 0.003. The model relies on the markers (templates), not on the policy keywords.

Concealed vulnerability per policy: Every RAI policy is detected with a recall of ≥ 0.93 (weakest: RAI-06 Security, 0.933).

Negation gap (`negation_gap` / `negation_blind_count`)

Quantifies how many minimal pairs the model treats as identical – the Goal-3 finding, now used as a test.

The package is divided into `loader.py` (Goal-2 artefacts), `analysis.py` (cross-validation, calibration, error indices), `explain.py` (top terms, local explanation, ablation) and `weakness.py` (the checks mentioned above).

4.3 Test coverage ($\geq 80\%$)

The `tests/test_xai.py` suite combines fast, deterministic unit tests with integration tests against the actual Goal 2 artefacts. These are skipped cleanly if the artefacts are missing. Error paths are explicitly checked (`KeyError` if a column is missing, `AttributeError` if the model is non-linear), which increases coverage beyond the simple if-statements.

Command: `python -m pytest --cov=xai --cov-report=term-missing`

Module	Statements	Missing	Coverage
xai/analysis.py	33	0	100%
xai/explain.py	52	0	100%
xai/weakness.py	89	3	97%
xai/loader.py*	38	3	92%
xai/__init__.py	6	0	100%
TOTAL	218	6	97%

Table 14: Goal 3 test coverage (28 passed, 1 xfailed)

*Note: measured with the Goal-2 artifacts present; loader.py reaches 92 % and the total 97 %.

4.4 One source of truth: notebook imports the tested package

The Goal-3 notebook does not duplicate the weakness logic inline. Its cells import `xai.weakness` and call the tested functions (`evasion_recall`, `per_policy_recall` / `worst_policy`, `mask_terms` / `obfuscate`, `negation_blind_count`).

See also: **GOAL3_Autonomous_Compliance_Sentinel.ipynb** | **Goal3_FIN.pptx**

5. Model improvement experiments and methods at the data level

The delivered model achieves a recall of 0.925 on the test set (precision 0.939, F1 0.932, ROC-AUC 0.991) at a threshold of 0.45. However, the analysis in Goal 3 has shown that this figure does not measure ethical violations, but rather the detection of text templates (markers) from the data generator. There is a label leak.

The following section explains the approaches we used to try to improve the baseline model (TF-IDF + logistic regression), the results we obtained, and the conclusions we drew from them. Ultimately, we identified five specific methods that focus on the data level (Goal 1). This is the starting point for identifying the actual root cause.

We counted the frequency of the strongest marker in the raw data (proposals_1000_EN.csv):

Group	Number of red flags	with "This requirement is NOT met"
grounded / real NFR (grounded_flag=1)	116	116 = 100.0 %
synthetic (grounded_flag=0)	217	0 = 0.0 %
Compliant (all)	667	0 = 0.0 %

Table 15: The marker is a perfect but only partial give-away signal

Wherever the marker appears, it is a red flag. The grounded cases all contain the sentence marker “This requirement is NOT met:” (coefficient 2.577, the strongest individual feature). The synthetic cases, by contrast, contain short template sentences, e.g. “Decisions cannot be traced or explained.” or “The target group is selected by gender.”

Experiment 1: different vectorisation and tokenisation

The GOAL3_Autonomous_Compliance_Sentinel.ipynb notebook was used to test whether a different text representation (different n-gram ranges, POS tokeniser instead of the standard tokeniser) would force the model to learn genuine content rather than the markers. Tested at a threshold of 0.40.

Configuration	Precision	Recall	F1	ROC-AUC
Baseline (stop_words, 1-2)	0.855	0.970	0.909	0.991
Raw ngram (5,6)	0.667	0.985	0.795	0.966
POS ngram (1,2)	0.882	1.000	0.937	0.996
POS ngram (2,3)	0.793	0.970	0.872	0.990
POS ngram (3,3) - chosen	0.780	0.955	0.859	0.987

Table 16: Five vectorisation experiments (threshold 0.40)

Result:

In terms of metrics, there is no real improvement. The chosen POS (3,3) configuration is actually weaker than the baseline (0.909), with an F1 score of 0.859. It was chosen because of the 3-word phrases, which are easier to read and explain for both a human reviewer and the agent.

The shortcut remains dominant in every configuration. In the selected variant, the marker coefficient even rises to 2.894 (“is not met” / “requirement is not”). This is accompanied by the synthetic template sentences (“group is selected by gender”, “decisions cannot be traced”).

The 'Raw ngram (5,6)' variant actively degrades the model due to vocabulary sparsity (precision 0.667) and is itself identified as a failure in the notebook.

Experiment 2: semantic embeddings instead of TF-IDF

In Part 4 of the Goal-2 Notebook, TF-IDF was replaced by BGE-small embeddings (Sentence Transformer) and classified using XGBoost.

The idea behind this is that semantic features, rather than literal tokens, might bypass the template shortcut.

Model	Recall	Precision	F1	ROC-AUC
TF-IDF + logistic regression (primary)	0.925	0.939	0.932	0.991
TF-IDF + XGBoost	0.970	0.956	0.963	0.997
BGE-small embeddings + XGBoost (experiment)	0.657	0.710	0.682	0.868

Table 17: Alternative models compared

Result:

With embeddings, the recall drops to 0.657. As soon as the literal marker tokens are no longer directly available as features, there is hardly any signal left. This is already an indication that the problem lies not in the text representation, but in the data itself.

Interim conclusion: what doesn't help

All model-based approaches fail at the same point: the signal is missing from the data:

- Tokeniser/n-gram tuning: Markers remain dominant (coefficient up to 2.894)
- Embeddings instead of TF-IDF: Recall drops to 0.657
- Lowering the threshold: simply swaps false negatives for false positives, without introducing any new signals
- Different classifier (XGBoost etc.) learns the same shortcut, just packaged differently

The real key lies in data generation (Goal 1).

Five effective methods at data level (Goal 1)

The following five methods create a data foundation from which a model can learn genuine understanding rather than mere patterns.

Method 1: Incorporate non-conformities into the requirements text rather than listing them separately

Currently, the violation is generated as a fixed, appended sentence. Instead, the problematic aspect should form part of the actual requirement, so that there is no separable marker sentence (which the model can learn by heart in isolation).

Method 2: Paraphrased variance per policy (data augmentation)

Instead of a single set of templates per policy, many different formulations of the same violation should be generated. This forces the model to focus on meaning rather than on the exact token.

Method 3: No tell-tale prefix in 'grounded' cases

The prefix "This requirement is NOT met:" makes grounded non-conformities too obvious and should be omitted. The non-conformity must be apparent from the content of the requirement, not from a label placed before it.

Method 4: Allow implicit and subtle breaches

Real-world suggestions rarely explicitly identify a violation. Some of the red flags should simply describe the problematic situation without labelling it. This forces the model to learn to assess meaning rather than simply counting keywords.

Method 5: A separate test set comprising real, marker-free suggestions

A small hold-out set comprising genuine or handwritten suggestions measures the true generalisation. This is the only way to demonstrate whether the system really works.

NOTE: ALL PROCESSES UP TO VERSION 2 ARE STORED / ARCHIVED IN THE 'version_1' PROJECT DIRECTORY.

5. Implementation Version 2

Previously, the cause of the 'label leak' was attributed to the data. In Version 2, the following changes have been implemented: a label-free training dataset containing rephrased, embedded violations (50 formulations per guideline, implicit cases and a neutral common context sentence). The model was retrained using this dataset and re-analysed using the same Goal-2 and Goal-3 pipelines. Although the following figures are lower, they are still meaningful: recall remains largely unchanged, whilst precision decreases. This is because the model can no longer rely on the template markers. The 'leak' has been eliminated (ablation drop of 0.0) and the model now relies on actual vocabulary. The remaining weaknesses (negation blindness) are data-independent.

Results on the optimised (marker-free) dataset

Metric / finding	Version 1 (with markers)	Version 2 (marker-free)
Primary model (LogReg) recall @ 0.45	0.925	0.851
Primary model precision @ 0.45	0.939	0.671
Primary model F1 @ 0.45	0.932	0.750
5-fold CV recall - LogReg	0.964	0.937
5-fold CV recall - XGBoost	0.985	0.823
Label-leak marker 'met' coefficient	2.577	none (removed)

Ablation recall (marker family removed)	0.991 to 0.586	0.985 (no drop)
Weakest RAI policy	RAI-06 Security (0.933)	RAI-08 Data Minimization (0.833)
Grounded / synthetic recall gap (R4)	0.009	0.227
Negation blind pairs (baseline)	5/5	5/5
xai test coverage	91%	97%

Table 18: Version 1 (original) vs. Version 2 (marker-free)

R4 gap (0.009 to 0.227):

The widening of the R4 gap from 0.009 to 0.227 is not a regression, but a sign that the test is working. The value of Version 1, which was close to zero, was an artefact of the marker, which assigned the same abbreviation to every ‘grounded’ red flag and obscured the actual difference. As soon as the marker is removed, the true, data-dependent gap becomes apparent (grounded recall of 1.000 versus a synthetic 0.773) and the test measures what it is intended to measure

6. Week 4 | Goal 4: The Autonomous Compliance Sentinel Agent

The agent builds directly on the three previous goals. It does not train anything new but instead utilises the results of those goals. The following table shows what the agent takes from each goal and via which file or interface this takes place.

Goal	What it delivers	How the agent uses it
Goal 1	The dataset (proposals_1000_EN.csv) and the RAI catalogue (RAI-01 ... RAI-09 with example sentences and severity).	The data is read indirectly via the Goal-2 loader. The RAI catalogue is stored in agent/policies.py and forms both the RAG knowledge base and the fixed severity rules.
Goal 2	The trained model (model_tfidf_logreg.joblib) and the hold-out predictions (test_predictions_logreg.csv) in the artifacts/ folder.	agent/loader.py loads exactly these files from the sibling folder goal2. The model is the gatekeeper (classify step).
Goal 3	The xai package: local explanation (tfidf · coefficient) and the weakness tests (negation, per-policy recall, ablation).	The trigger-word logic sits in the agent (classify.py). The Extended version additionally imports goal3/xai directly to certify the model before use (certify.py).
Goal 4	The agent itself (this package).	Orchestrates all the sources above as a LangGraph state machine.

Table 19: Goals

Architecture of the agent, the LangGraph state machine

The agent is a real LangGraph StateGraph.

Node	Task	Source involved
classify	Compute the Red-Flag probability and reduce it to 0/1 at the 0.40 threshold.	Goal-2 model
assess	Get trigger words, retrieve all matching RAG policies, determine the worst severity, phrase the verdict.	Goal-3 XAI + RAG + fixed rules + LLM
advise (Extended only)	Consult the LLM for a policy angle a human should double-check; advisory only, does not change the decision; shown only when relevant.	advisor.py + Groq LLM
human_gate	High → a human decides (APPROVE/REJECT); Medium → automatic. Write an audit entry.	Human-in-the-loop + audit
compliant	Proposal is not a Red Flag → end, no action.	–
cleared (Extended only)	The verdict concluded "no real violation" → end without a ticket.	Open-ended verification
writeback (Extended only)	Write a Confluence comment, a Jira ticket and an audit log to the outbox folder.	Writeback

Table 20: Nodes

Step by step, a proposal runs through the agent

This is how the agent processes a single proposal text (in brackets is the pipeline component that performs each step).

1. Input: the proposal text enters the state as a string (proposal_text). In batch mode it is read from a file in the inbox folder.
2. classify (Goal-2 model): the loaded tfidf+logreg model computes the Red-Flag probability (predict_proba). If it is ≥ 0.40 , y_pred = 1 (Red Flag), otherwise 0 (compliant).
3. Branch: if y_pred = 0, it goes straight to the compliant node and the agent ends with "no action". Only for y_pred = 1 does the expensive part follow.
4. assess trigger words (Goal-3 XAI): for the text the strongest words are computed by tfidf value · coefficient. This grounds the later reason in real model evidence.
5. assess RAG search (semantic): a query is built from the strongest trigger words and run against all nine RAI policies. Not only the top hit is returned but every policy whose score is close to the best hit (default margin 0.06). A document that violates several policies matches several of them almost equally. They are all listed as matched_policies.
6. assess fixed rule (severity): for every matched policy the fixed severity is looked up (RAI-01...05 = High, RAI-06...09 = Medium). What governs the onward path is the worst severity among the matches (governing severity): as soon as one High is present, High applies. This way a single Medium top hit can no longer hide a High violation.
7. assess verdict (LLM or offline): from the proposal text, trigger words and policy text the LLM phrases two sentences: reason and suggested fix – on that basis only. The reason is phrased around the worst matched policy (with several High, the one with the best score) so that reason and governing severity match. Without a Groq key a deterministic template does the phrasing. (Extended: here it is additionally decided whether there really is a violation – is_violation.)
The LLM used is Groq's llama-3.1-8b-instant (temperature 0.0).
8. assess findings per policy: for a multi-violation, every matched policy gets its own finding (reason + fix). The worst policy keeps the main verdict (possibly from the LLM), the other findings are generated from the offline template. This way not just one case is written out but every one – in the CLI, the UIs, the Confluence comment and the Jira ticket.
9. Transparency note: the verdict carries a provenance note. It states clearly that the Red-Flag decision comes from the trained TF-IDF model, while the reason and fix were phrased by an LLM and may contain errors (or come from a fixed template if no LLM was active). This sentence appears in the CLI, the UIs, the Confluence comment and the Jira ticket. For a multi-policy Red Flag the primary finding is phrased by the LLM while the additional policies use a fixed template, so the note now names both sources explicitly.
10. advise (LLM hint, second tool): the LLM is consulted once for a policy angle a human reviewer should personally double-check – for example a policy the automatic match under-weighted. It is advisory only: it does not change the prediction, severity or findings, and a hint is shown only when the LLM raises something relevant. Without a Groq key nothing is shown.
11. human_gate: for High severity a human is asked to APPROVE/REJECT (in the terminal or via the UI). For Medium it is approved automatically. In both cases an audit entry is created with prediction, probability, policy, severity and trigger words (Article 12).
12. Finish (Extended – writeback): a Confluence comment and – for a real High violation – a Jira ticket are written to the outbox folder, plus a line in the machine-readable audit log agent_log.jsonl.

Features implemented

Option	File / location	What it does
Weakness pre-gate	agent/certify.py	Imports goal3/xai and checks the model before use: negation robustness and weakest-policy recall. Returns passed = True/False.
Real writeback	agent/writeback.py → outbox/	Writes a Confluence comment, a Jira ticket (only for a real High violation) and a line into the audit log agent_log.jsonl.
Batch processing	agent/batch.py, run_batch.py, inbox/	Processes every *.txt file in the inbox folder and prints a summary table.
Configurable severity	config/policies.csv	Overrides the built-in High/Medium mapping if the file is present.
Borderline LLM routing	agent/graph.py (llm_band)	The LLM is only used inside the uncertain probability band (default 0.35–0.65); clear cases use the offline template – saving LLM calls.
Open-ended verification	agent/graph.py + agent/llm.py (is_violation)	The agent may classify a false alarm as "no violation" instead of assuming a violation.
ipywidgets UI	agent/ui.py	Text box + Assess button in the notebook; ui_available() secures a fallback.
Streamlit web UI	streamlit_app.py	streamlit run streamlit_app.py opens a local web page with a two-column layout (interface left, charts right); the human gate becomes real APPROVE/REJECT buttons.
PDF upload	agent/pdf.py + streamlit_app.py	Instead of typing text, a proposal can be uploaded as a PDF. extract_text reads the text (pdfplumber, fallback pypdf) and fills the input field. Text PDFs only; scanned PDFs would need OCR.
Analysis charts (A1–A3)	charts.py + streamlit_app.py	For a Red Flag the Streamlit UI shows three Matplotlib charts on the right: A1 trigger words (word contributions), A2 RAG match score per policy (with severity colours and a match cutoff) and A3 Red-Flag probability against the threshold. Theme-aware (light/dark).
LLM policy advisor (second tool)	agent/advisor.py	Consults the LLM for a policy angle a human should double-check; shown only when relevant; advisory, never changes the decision.

Table 21: Features

Offline and fallback behaviour

The agent runs fully offline.

If missing	Fallback	How to
langchain-groq or GROQ_API_KEY	A deterministic text template phrases reason and fix.	groq_available() == False; verdict['source'] == 'offline'
Embedding model / langchain-chroma	TF-IDF cosine search over the nine policy texts.	PolicyRetriever.mode == 'keyword'
ipywidgets	The notebook uses run_once() or the Streamlit UI directly.	ui.ui_available() == False

Note: If the Groq key is set and the extra packages are installed, the agent switches automatically to the real LLM and semantic.

See also: **GOAL4_Autonomous_Compliance_Sentinel.ipynb | Pipeline goal 4**

List of tables

Table 1: Datasets	7
Table 2: The policy catalogue.	7
Table 3: Characteristics of proposals_1000_EN.csv	9
Table 4: Project plan.....	10
Table 5: Regulatory analysis	10
Table 6: Setup.....	11
Table 7: Results on the test set	11
Table 8: Recall per policy.....	12
Table 9: Group – GAP 0.009.....	12
Table 10: issue type – GAP 0.085	12
Table 11: 7 Groups	12
Table 12: 7 The risk list as defined in Week 2.....	13
Table 13: Alternative models tested in Week 2	13
Table 14: Goal 3 test coverage (28 passed, 1 xfailed)	15
Table 15: The marker is a perfect but only partial give-away signal	16
Table 16: Five vectorisation experiments (threshold 0.40).....	16
Table 17: Alternative models compared	17
Table 18: Version 1 (original) vs. Version 2 (marker-free)	19
Table 19: Goals	20
Table 20: Nodes	20
Table 21: Features.....	22
Table 22: Fallback	23